

COMS 4170 Software Testing Spring 2026

Automated Testing Tools

1

Selenium IDE

<https://www.selenium.dev/>

Selenium automates browsers. That's it!
What you do with that power is entirely up to you.

Primarily it is for automating web applications for testing purposes, but is certainly not limited to just that.
Boring web-based administration tasks can (and should) also be automated as well.

2

Selenium
IDE

Selenium provides automation for
web testing

Manually record tests and then
replay them

Or you can use their programming
interface

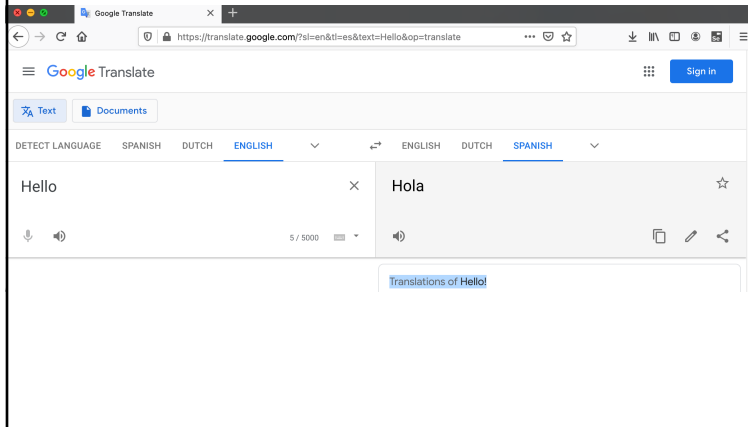
3

Selenium Does not Generate Tests However

That is up to you...

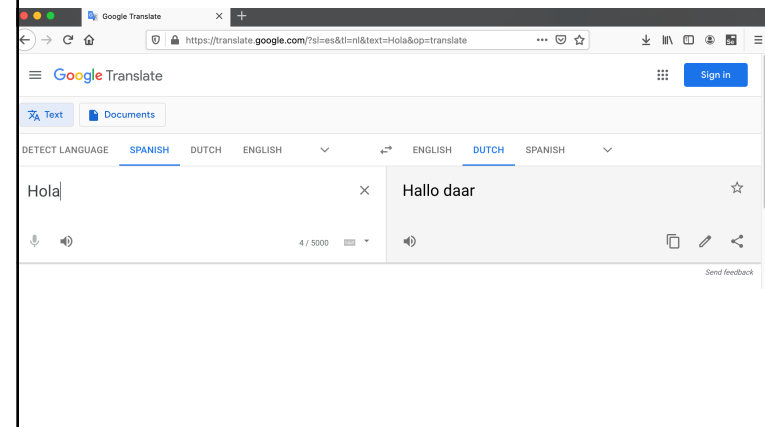
4

Some Motivation



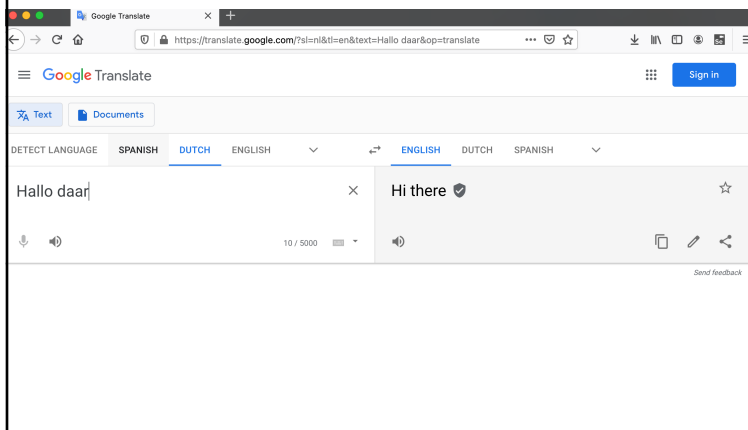
5

Some Motivation



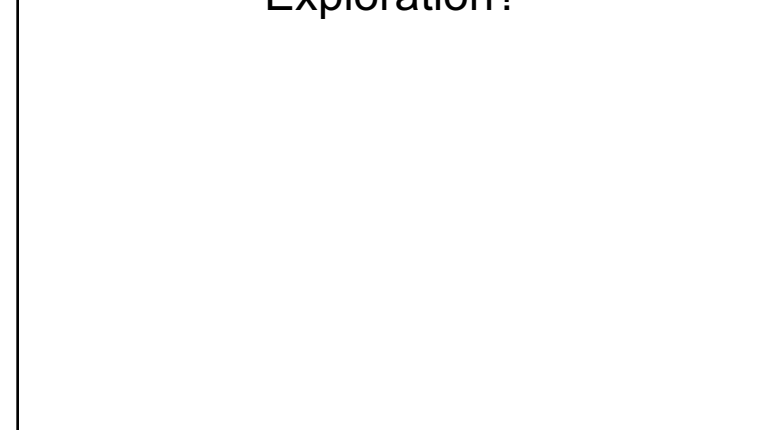
6

Some Motivation



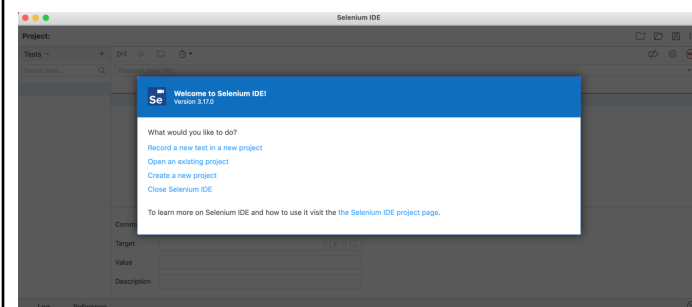
7

How can We Automate Exploration?



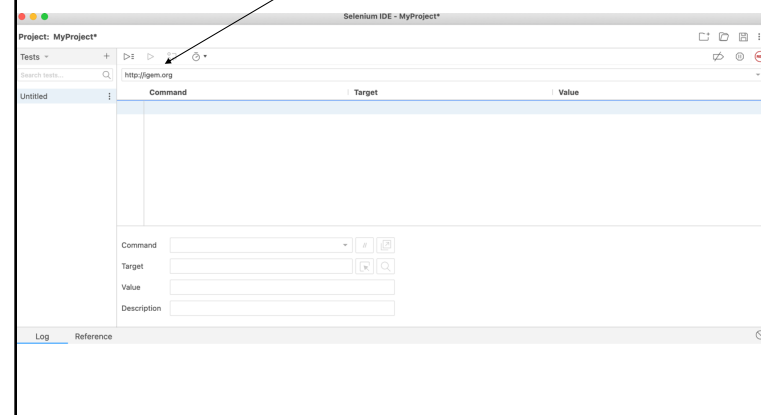
8

Starting Selenium



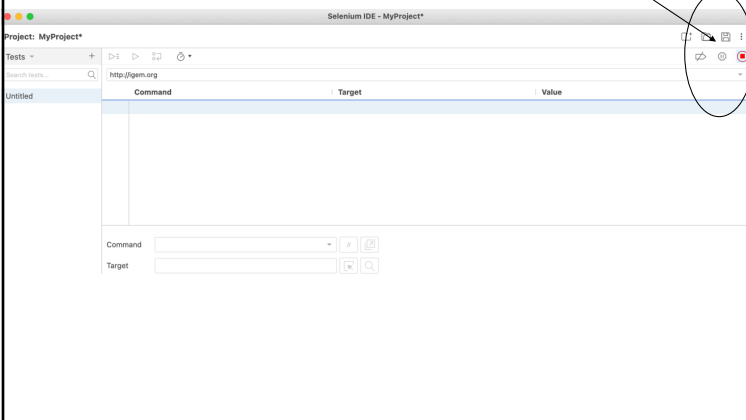
9

Base URL



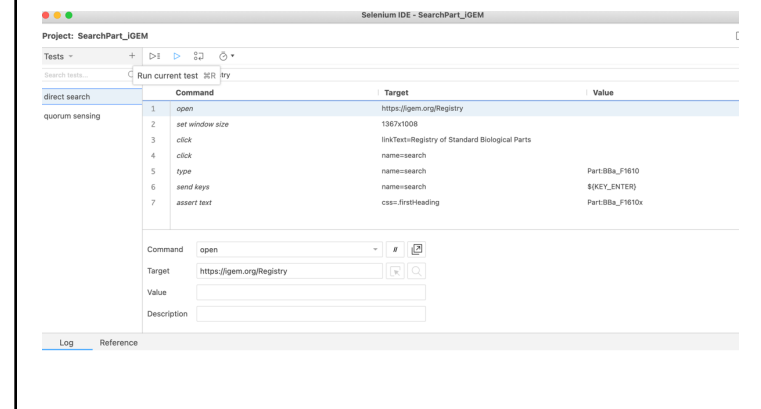
10

Recording



11

Replaying Test



12

After It is Run

Project: SearchPart_IGEM

Executing -

direct search

	Command	Target	Value
1	open	https://igem.org/Registry	
2	set window size	1367x1008	
3	click	link/Registry of Standard Biological Parts	
4	click	name-search	
5	type	name-search	Part:BBa_F1810
6	send keys	name-search	\$KEY_ENTER
7	assert text	css:firstHeading	Part:BBa_F1810a

Command:

Target:

Value:

Description:

Run: 1 - Failed: 1

Log Reference

Running 'direct search'

1. open on https://igem.org/Registry OK
2. setWindow size on 1367x1008 OK
3. click on link/Registry of Standard Biological Parts OK
4. Trying to find name-search... OK
5. type on name-search with value Part:BBa_F1810 OK
6. sendkeys on name-search with value \$KEY_ENTER OK
7. assertText on css-firstHeading with value Part:BBa_F1810a Failed
Actual value "Part:BBa_F1810" did not match "Part:BBa_F1810a"

'direct search' ended with 1 error(s)

13

For this Demo

1. **Record/Replay** a query on the igem.org/parts website using selenium
2. Validate (using Assert text) that result is correct
3. **Save project** and examine contexts
4. **Create and run a set of tests** for a google forms

14

The .side File

```
{
  "id": "ba45c7a5-855a-4bdd-8e3c-f20bd7c1fd19",
  "version": "2.0",
  "name": "SearchPart_IGEM",
  "url": "https://igem.org/Registry",
  "tests": [
    {
      "id": "67146974-c1a4-46b9-a865-d7021798d224",
      "name": "quorum sensing",
      "commands": [
        {
          "id": "7dc4cb81-4498-4436-b295-f43cf9e12731",
          "comment": "",
          "command": "open",
          "target": "/Registry",
          "targets": [],
          "value": ""
        }
      ],
      "id": "b3ba815b-4e4f-45b7-9b8f-bf2771963edd",
      "comment": "",
      "command": "setWindowSize",
      "target": "1138x828",
      "targets": [],
      "value": ""
    }
  ]
}
```

15

Test Generation Tools

- We have talked about automation of tests
- But what about automation of test cases?

16

Randoop

Random unit test generator

- <https://randoop.github.io/randoop/>



17

Evosuite



<https://www.evosuite.org>

18

Idea

- Uses a **search algorithm** to find test cases that cover as much code as possible
- **Creates Junit test** cases
- Builds **oracles from the code** itself
- Other Java test generation tools:
 - Randoop

19

Using Evosuite

- I will put a file for you in canvas [evosuite-S25.tar](#)
- It has the .jar files and 2 example programs
- It **runs on pyrite (need Java 11 -or 8)**
`/usr/lib/jvm/java-11/bin/java`
`/usr/lib/jvm/java-1.8.0/bin/java`
- There are also maven and docker versions (your choice)

20

Demo

```
[mcohen@pyrite-n3 evosuite]$ ls
evosuite-1.1.0.jar  evosuite-standalone-runtime-1.1.0.jar  lib      Tutorial_Stack
evosuite-report     evosuite-tests                             Triangle
[mcohen@pyrite-n3 evosuite]$

[mcohen@pyrite-n3 evosuite]$ ls
evosuite-1.1.0.jar  evosuite-standalone-runtime-1.1.0.jar  lib      Tutorial_Stack
evosuite-report     evosuite-tests                             Triangle
[mcohen@pyrite-n3 evosuite]$ javac Tutorial_Stack/tutorial/*.java
Note: Tutorial_Stack/tutorial/Stack.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
[mcohen@pyrite-n3 evosuite]$ javac Triangle/triangle/*.java
[mcohen@pyrite-n3 evosuite]$
```

21

Demo

```
[mcohen@pyrite-n3 evosuite]$ ls
evosuite-1.1.0.jar  evosuite-standalone-runtime-1.1.0.jar  lib      Tutorial_Stack
evosuite-report     evosuite-tests                             Triangle
[mcohen@pyrite-n3 evosuite]$ javac Tutorial_Stack/tutorial/*.java
Note: Tutorial_Stack/tutorial/Stack.java uses unchecked or unsafe operations.
[mcohen@pyrite-n3 evosuite]$ java -jar evosuite-1.1.0.jar -projectCP Triangle -class triangle.TriangleType
* EvoSuite 1.1.0
* Going to generate test cases for class: triangle.TriangleType
* Starting Client-0
* Connecting to master process on port 17649
* Analyzing classpath:
  - Triangle
* Finished analyzing classpath
* Generating tests for class triangle.TriangleType
* Test criteria:
  - Line Coverage
  - Branch Coverage
  - Exception
  - Mutation testing (weak)
  - Method-Output Coverage
  - Top-Level Method Coverage
  - No-Exception Top-Level Method Coverage
  - Context Branch Coverage
[Progress:>] 0% [Cov:>] 0% * Total number of test g
oals for DYNAMOSA: 173
* Using seed 1613833653736
* Starting evolution
* Initial Number of Goals in DynaMOSA = 13 / 173
[Progress:>] 3% [Cov:===== 92%]
```

22

Demo

```
* Total number of goals: 2
* Number of covered goals: 2
* Coverage analysis for criterion CBRANCH
* Coverage of criterion CBRANCH: 96%
* Total number of goals: 23
* Number of covered goals: 22
* Generated 19 tests with total length 19
* Resulting test suite's coverage: 93% (average coverage for all fitness functions)
* Generating assertions
* Resulting test suite's mutation score: 85%
* Compiling and checking tests
* Writing tests to file
* Writing JUnit test case 'TriangleType_ESTest' to evosuite-tests
* Done!

* Computation finished
[mcohen@pyrite-n3 evosuite]$
```

23

Setting Classpath

- Needed for running unit tests

```
[mcohen@pyrite-n3 evosuite]$ export CLASSPATH=.evosuite-tests:lib/junit-4.13.jar:lib/hamcrest-core-1.3.jar:lib
:TriangleTutorial_Stack:evosuite-standalone-runtime-1.1.0.jar
[mcohen@pyrite-n3 evosuite]$
```

24

Compiling/Running Tests

```
[mcohen@pyrite-n3 evosuite]$ export CLASSPATH=.:evosuite-tests:lib/junit-4.13.jar:lib/hamcrest-core-1.3.jar:lib/
:Triangle:Tutorial_Stack:evosuite-standalone-runtime-1.1.0.jar
[mcohen@pyrite-n3 evosuite]$ javac evosuite-tests/triangle/*.java
[mcohen@pyrite-n3 evosuite]$ java org.junit.runner.JUnit4 triangle.TriangleType_ESTest
JUnit version 4.13
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/mcohen/COMS_417_demo/evosuite
/evosuite-standalone-runtime-1.1.0.jar) to field java.awt.GraphicsEnvironment.headless
WARNING: Please consider reporting this to the maintainers of org.evosuite.runtime.GuiSupport
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
.....
Time: 6.175
OK (19 tests)
[mcohen@pyrite-n3 evosuite]$
```

25

The Tests

```
public class Stack_ESTest extends Stack_ESTest_scaffold

@Test(timeout = 4000)
public void test0() throws Throwable {
    Stack<Object> stack0 = new Stack<Object>();
    assertTrue(stack0.isEmpty());

    Integer integer0 = new Integer((-20));
    stack0.push(integer0);
    stack0.push(integer0);
    stack0.pop();
    assertFalse(stack0.isEmpty());
}

@Test(timeout = 4000)
```

26

Fuzzing (security)

- Used to detect program crashes/memory leaks for security

American fuzzy lop (AFL)

- <https://github.com/google/AFL>

27

AFL

2) The afl-fuzz approach

American Fuzzy Lop is a brute-force fuzzer coupled with an exceedingly simple but rock-solid instrumentation-guided genetic algorithm. It uses a modified form of edge coverage to effortlessly pick up subtle, local-scale changes to program control flow.

Simplifying a bit, the overall algorithm can be summed up as:

1. Load user-supplied initial test cases into the queue,
2. Take next input file from the queue,
3. Attempt to trim the test case to the smallest size that doesn't alter the measured behavior of the program,
4. Repeatedly mutate the file using a balanced and well-researched variety of traditional fuzzing strategies,
5. If any of the generated mutations resulted in a new state transition recorded by the instrumentation, add mutated output as a new entry in the queue.
6. Go to 2.

The discovered test cases are also periodically culled to eliminate ones that have been obsoleted by newer, higher-coverage finds; and undergo several other instrumentation-driven effort minimization steps.

As a side result of the fuzzing process, the tool creates a small, self-contained corpus of interesting test cases. These are extremely useful for seeding other, labor- or resource-intensive testing regimes - for example, for stress-testing browsers, office applications, graphics suites, or closed-source tools.

The fuzzer is thoroughly tested to deliver out-of-the-box performance far superior to blind fuzzing or coverage-only tools.

28

Fuzzing A Triangle Program

```

/*
 * C program to check whether a triangle is Equilateral, Isosceles or Scalene
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main()
{
    int side1, side2, side3;
    char name[50];
    /* Input sides of a triangle */
    /* printf("Enter three sides of triangle: ");
    scanf("%d%d%d", &side1, &side2, &side3);
    if(side1 <= 0 || side2 <= 0 || side3 <= 0)
        strcpy(name, "Invalid");

    if(side1 == side2 && side2 == side3 || side2 == side3 && side1 == side2 || side1 == side3 && side2 == side1)
        strcpy(name, "Equilateral");

    if(side1 == side2 || side1 == side3 || side2 == side3)
    {
        /* If all sides are equal */
        strcpy(name, "This is an Equilateral Triangle");
    }
    else if(side1 == side2 || side1 == side3 || side2 == side3)
    {
        /* If any two sides are equal */
        strcpy(name, "Isosceles");
    }
    else
    {
        /* If none sides are equal */
        strcpy(name, "Scalene");
    }
    printf("%s\n", name);
    return 0;
}

```

29

Fuzzing A Triangle Program

american fuzzy lop 2.57b (triangle)			
process timing		overall results	
run time : 0 days, 0 hrs, 0 min, 15 sec		cycles done : 0	
last new path : 0 days, 0 hrs, 0 min, 14 sec		total paths : 16	
last uniq crash : 0 days, 0 hrs, 0 min, 9 sec		uniq crashes : 1	
last uniq hang : none seen yet		uniq hangs : 0	
cycle progress		map coverage	
now processing : 10 (62.50%)		map density : 0.02% / 0.04%	
paths timed out : 0 (0.00%)		count coverage : 1.00 bits/tuple	
stage progress		findings in depth	
now trying : havoc		favored paths : 8 (50.00%)	
stage execs : 980/1024 (95.70%)		new edges on : 10 (62.50%)	
total execs : 8314		total crashes : 1 (1 unique)	
exec speed : 545.6/sec		total tmouts : 2 (1 unique)	
fuzzing strategy yields		path geometry	
bit flips : 4/224, 2/220, 1/212		levels : 2	
byte flips : 0/28, 0/24, 0/16		pending : 13	
arithmetics : 1/1560, 0/348, 0/0		pend fav : 5	
known ints : 0/136, 0/656, 0/704		own finds : 8	
dictionary : 0/0, 0/0, 0/0		imported : n/a	
havoc : 1/3072, 0/0		stability : 100.00%	
trim : 0.00%/4, 0.00%		[cpu000: 29%]	

30